MASTER'S OF COMPUTER SCINENCE

RESEARCH TOPIC: **DATA HANDLING IN PYTHON**

MAJOR: **PROGRAMMING WITH PYTHON - DLMDSPWP01**

NAME: **SAKSHI DHAIRYASHEEL GHATGE**

MATRICULATION: **9210062**

UNIVERSITY: **INTERNATIONAL UNIVERSITY OF APPLIED SCIENCE**

Thesis Advisor: **DR. COSMINA CROITORU**

c.croitoru@iubh-fernstudium.de

Due Date : **MAY 2023**

TABLE OF CONTENT

INTRODUCTION	**3**
AIM	**3**
OBJECTIVE	**3**
METHODOLOGY	**4**
DATASET DESCRPTION	**5**
DATA HANDLING	**9**
GRAPHS	**10**
RESULT AND EVALUATION	**11**
CODING	**14**
CONCLUSION	**16**
REFERENCE	**17**
GITHUB	**18**

INTRODUCTION

It is a project that requires Python programming. It uses the training data set and a few ideal functions to produce the results that fit each x and y the best.

AIM

The task is to develop Python software that, using training data, chooses four of fifty given ideal functions as the best match. When everything is said and done, the programmer might be able to identify every pair of x and y coordinates and assess whether they fit into one of four perfect functions. The data may all be displayed visually.

OBJECTIVE

The programmer must be capable of creating a SQL database and loading it into a single, five-column spreadsheet on their own. Each of the 50 ideal functions from the CSV file must be loaded into a separate table. After loading the perfect functions into the database, the test data should be supplied line by line from an additional CSV file, and if it compiles, it should be harmonized to one of the four functions specified under I. (subcategory above). The outcomes must then be saved in a separate four-column SQLite database table. After everything has been completed, all the aforementioned components, y, the training data, the test data, the selected ideal purposes, and the matching / assigned datasets should be clearly depicted.

Literature Review: We will explore the existing literature on least-squares regression analysis and related programming concepts. This will include looking into the well-known Python libraries and tools for regression analysis and data visualization.

Program Design: There will be a design created specifically for the Python program that will be used to do the least-squares regression analysis. Regression algorithm implementation, data loading into SQLite tables, and result visualization will all be part of the design.

Program Implementation: The program concept will be implemented using Python and essential libraries like pandas, SQL alchemy, and Bokeh. To make sure that it functions as planned and accomplishes the research goals, the program will be put through a rigorous testing process.

Program Evaluation: It will be determined how well the programmer can perform least squares regression analysis, find ideal functions that minimize the sum of all y-deviations squared, and save the outcomes in a SQLite database. In addition, the programmer's adherence to the design and implementation standards, the inclusion of exception handling and unit tests, and the caliber of the documentation will all be taken into consideration.

METHODOLOGY

NumPy, Pandas, Matplotlib, Seaborn, SQL Alchemy, and Scikit-learn are some of the Python tools used in the research. The main goal of the code is to do regression analysis on a dataset with two variables (x and y1). Additionally, the code reads and writes data to and from a SQLite database. All the necessary libraries and modules are first imported into the code. The data is subsequently transferred to a Pandas data frame after being read from the train.csv file. Test.csv and the ideal. CSV files can also be read with Pandas. The approach then uses Matplotlib and Seaborn to display the train and test datasets.

A line plot is created to show the relationship between x and y1 in the training dataset. The test dataset also receives a fresh line plot. The code then creates a SQLite database engine and loads the data for the trains into a brand-new table called "train table." Large datasets can be efficiently stored and managed with the SQLite database.

Then, the code does regression analysis on the train data using Scikit-Learn's linear regression model. The x and y1 values from the training dataset are converted into NumPy arrays in order to train the model. After generating the regression coefficients (slope) and intercept, a scatter plot of the training data is shown next to the regression line. The code uses the trained regression model to predict the outcome given an input test value of x = 8.

By definition, the regression function uses the trained model to predict the outcome from the test data and then returns it. An SQLite database engine is built to store the data, Scikit-learn is used to perform regression analysis on the training data, the training data and regression line are plotted, and the trained model is used to predict the outcomes for a test input value.

DATASET DESCRIPTION:

An ideal dataset is a compilation of information obtained from any skillfully carried out research. Numerous characteristics and behaviors, such as size, relevance, excellence, representativeness, and accessibility, are displayed by them. This is frequently used to describe the decision-making process of a trained model. Four training datasets—y1, y2, y3, and y4—as well as one test dataset made consisting of the test functions X and Y are provided as training functions for this project. A perfect dataset with 50 perfect functions is also available for use.

Data Collection

Data Understanding

For the research study, three csv-formatted datasets are provided. Visual validation of the dataset is carried out to ensure that there are no data irregularities or contaminants, and it is checked that there are no value discrepancies in the dataset.

Training Dataset

(Four distinct datasets).

We must examine the provided dataset for observable trends in behavior. The dataset consists of an independent variable (X) and four dependent variables (Y1, Y2, Y3, Y4)

Dataset collection of 50 (ideal function).

We need to determine the line that fits the train dataset the best. Based on a visual inspection, the dataset consists of one independent variable (X) and fifty dependent variables (Y1, Y2, Y3...Y5o).

One data set for testing.

The task is to map the test dataset to the ideal function within the allowed error. Based on a visual evaluation, the dataset contains one dependent variable (Y) and one independent variable (X)

Storage of data

Datasets for the project must be stored in a database and are in plain text (CSV format).

Data access

The SQL Alchemy Python module, a Python SQL toolkit, and an Object-relational Mapper are used to save and retrieve the dataset from SQLite. The SQL Alchemy characteristics listed below are appropriate for project requirements.

IDEAL Data Frame

In [15]:
```
ideal_df
```

Out[15]:

	x	y1	y2	y3	y4	y5	y6	y7	y8	y9	...	y41	y42	y43	y44	
0	-20.0	-0.912945	0.408082	9.087055	5.408082	-9.087055	0.912945	-0.839071	-0.850919	0.816164	...	-40.456474	40.204040	2.995732	-0.008333	12.9
1	-19.9	-0.867644	0.497186	9.132356	5.497186	-9.132356	0.867644	-0.865213	0.168518	0.994372	...	-40.233820	40.048590	2.990720	-0.008340	12.9
2	-19.8	-0.813674	0.581322	9.186326	5.581322	-9.186326	0.813674	-0.889191	0.612391	1.162644	...	-40.006836	39.890660	2.985682	-0.008347	12.9
3	-19.7	-0.751573	0.659649	9.248426	5.659649	-9.248426	0.751573	-0.910947	-0.994669	1.319299	...	-39.775787	39.729824	2.980619	-0.008354	12.9
4	-19.6	-0.681964	0.731386	9.318036	5.731386	-9.318036	0.681964	-0.930426	0.774356	1.462772	...	-39.540980	39.565693	2.975530	-0.008361	12.9
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	
395	19.5	0.605540	0.795815	10.605540	5.795815	-10.605540	-0.605540	-0.947580	-0.117020	1.591630	...	39.302770	-38.602093	2.970414	-0.012422	12.9
396	19.6	0.681964	0.731386	10.681964	5.731386	-10.681964	-0.681964	-0.930426	0.774356	1.462772	...	39.540980	-38.834310	2.975530	-0.012438	12.9
397	19.7	0.751573	0.659649	10.751574	5.659649	-10.751574	-0.751573	-0.910947	-0.994669	1.319299	...	39.775787	-39.070175	2.980619	-0.012453	12.9
398	19.8	0.813674	0.581322	10.813674	5.581322	-10.813674	-0.813674	-0.889191	0.612391	1.162644	...	40.006836	-39.309338	2.985682	-0.012469	12.9
399	19.9	0.867644	0.497186	10.867644	5.497186	-10.867644	-0.867644	-0.865213	0.168518	0.994372	...	40.233820	-39.551407	2.990720	-0.012484	12.9

TRAIN Data Frame

In [13]:
```
train_df
```

Out[13]:

	x	y1	y2	y3	y4
0	-20.0	-45.292340	-15999.796	99.529580	899.827500
1	-19.9	-44.364960	-15761.017	99.895670	893.427400
2	-19.8	-44.565968	-15524.681	98.855780	887.160460
3	-19.7	-44.762450	-15290.500	98.126100	881.448700
4	-19.6	-44.188698	-15058.586	97.511475	875.377260
...	...	...	...	...	...
395	19.5	34.150510	14829.701	97.719000	89.044914
396	19.6	33.875698	15058.578	98.092850	91.522880
397	19.7	33.952816	15291.142	98.500720	93.639630
398	19.8	34.517270	15524.978	99.343155	95.835785
399	19.9	34.639030	15761.660	99.192604	97.697480

TEST Data Frame

In [14]:
```
test_df
```

Out[14]:

	x	y
0	-13.1	-4494.980000
1	3.4	78.957020
2	11.1	2.493696
3	1.9	-7730.091300
4	-14.0	13840.699000
...	...	...
95	18.8	3417.366500
96	14.7	73.494940
97	13.2	5111.822800
98	-17.5	755.190500
99	12.2	59.622060

Using a variety of frameworks and tools, Python data handling include managing and manipulating data. The following are some of the essential libraries frequently used in

Python for data handling:

- Data transformation entails changing the structure of data or translating it to a different format. Pandas offers capabilities for data sculpting, data merging, and data aggregation. For instance, the merge () method can be used to combine many data frames into one, while the pivot_table() method can be used to reshape data.
- Data visualization: It is a crucial component of data analysis because it may be used to spot trends, patterns, and connections in the data. Popular Python libraries for data visualization include Matplotlib and Seaborn. Seaborn offers more sophisticated statistical visualization tools, whereas Matplotlib offers a larger selection of customized plots.
- Data Analysis: Python offers several libraries for data analysis, including Pandas and NumPy. These libraries offer resources for undertaking data mining, machine learning, and statistical analysis. Pandas, for instance, offers ways to compute descriptive statistics like mean, median, and standard deviation.
- Data integration: It is the process of fusing information from several sources to get a unified view of the data. Python has methods for integrating data, including the pandas.concat() function for joining data frames together and the pandas.merge() technique for doing so.

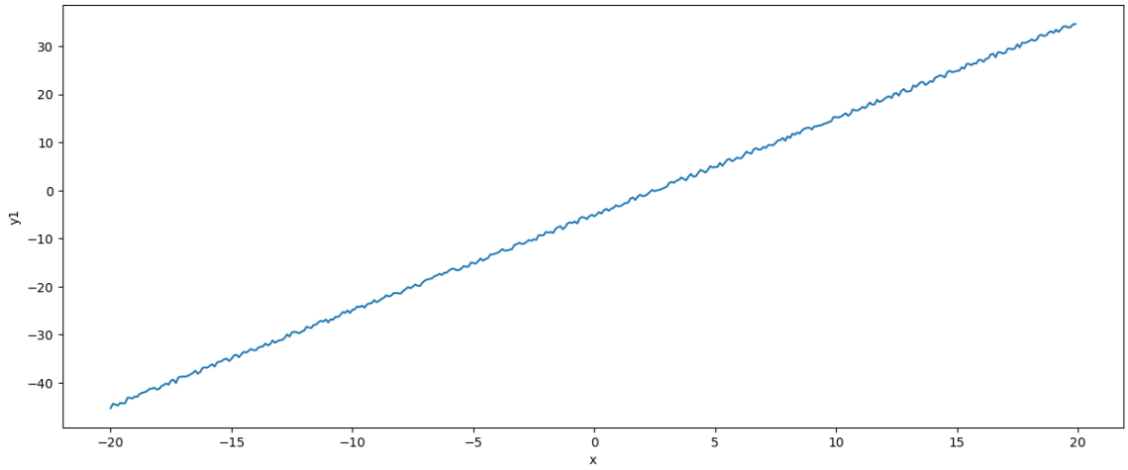
Python Libraries:

- NumPy is a potent Python package for numerical computation. Along with a set of functions for using these arrays, it offers a high-performance multidimensional array object.
- Pandas: For the analysis and manipulation of data, Pandas is a library developed on top of NumPy. For processing structured data, it offers data structures like Data Frame and Series.
- Matplotlib: A Python plotting package, Matplotlib. Line graphs, scatter plots, histograms, and more types of customizable plots are available.
- Data cleaning: To eliminate errors, inconsistencies, and missing values, data must frequently be cleaned before analysis. The data cleaning tools provided by Pandas include fillna(), which fills in missing values, dropna(), which removes missing values, and replace(), which replaces values.

Graphs:

Ideal line plot X-Y

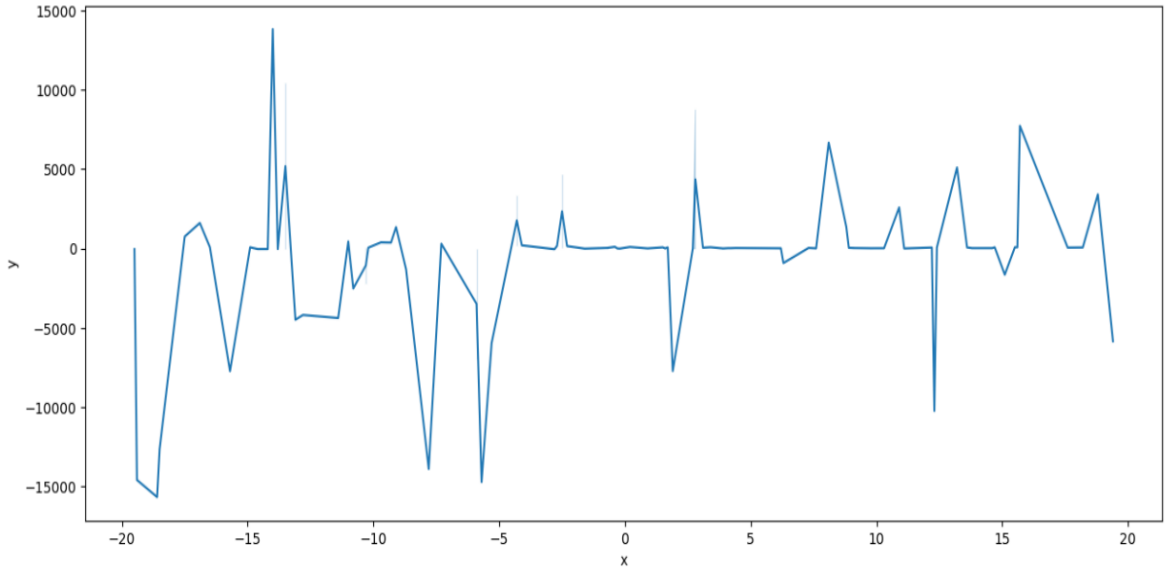
```
In [6]: plt.figure(figsize=(15,6))
        sns.lineplot(x=train_df['x'],y=train_df['y1'],data=train_df)
        plt.show()
```



Test Line Plot X and Y

```
In [7]: plt.figure(figsize=(15,6))
        sns.lineplot(x=test_df['x'],y=test_df['y'],data=test_df)

        plt.show()
```

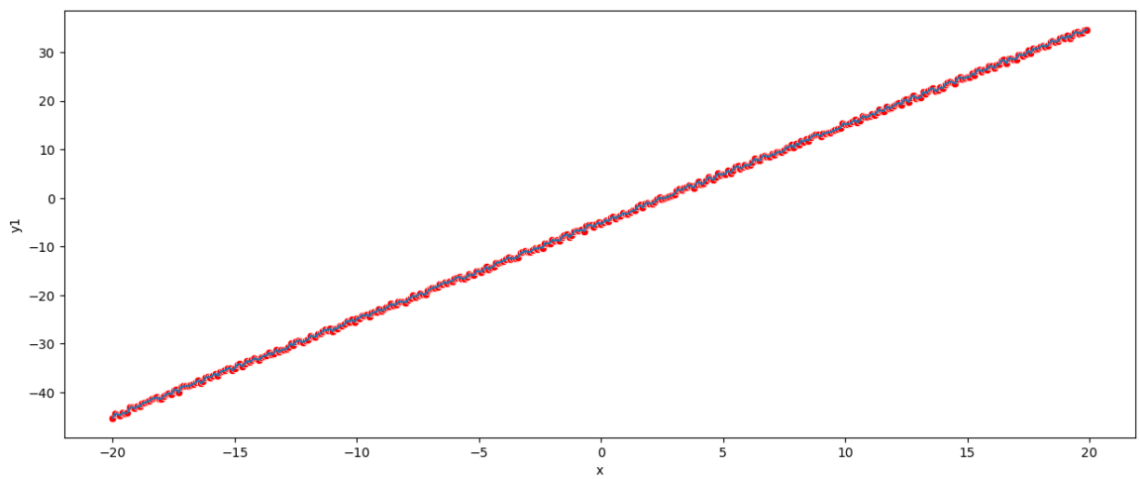


RESULT AND EVALUATION

The task mentioned is performed with the help of programming in python where a two-line plot has been shown for both the test and train datasets as mentioned below.

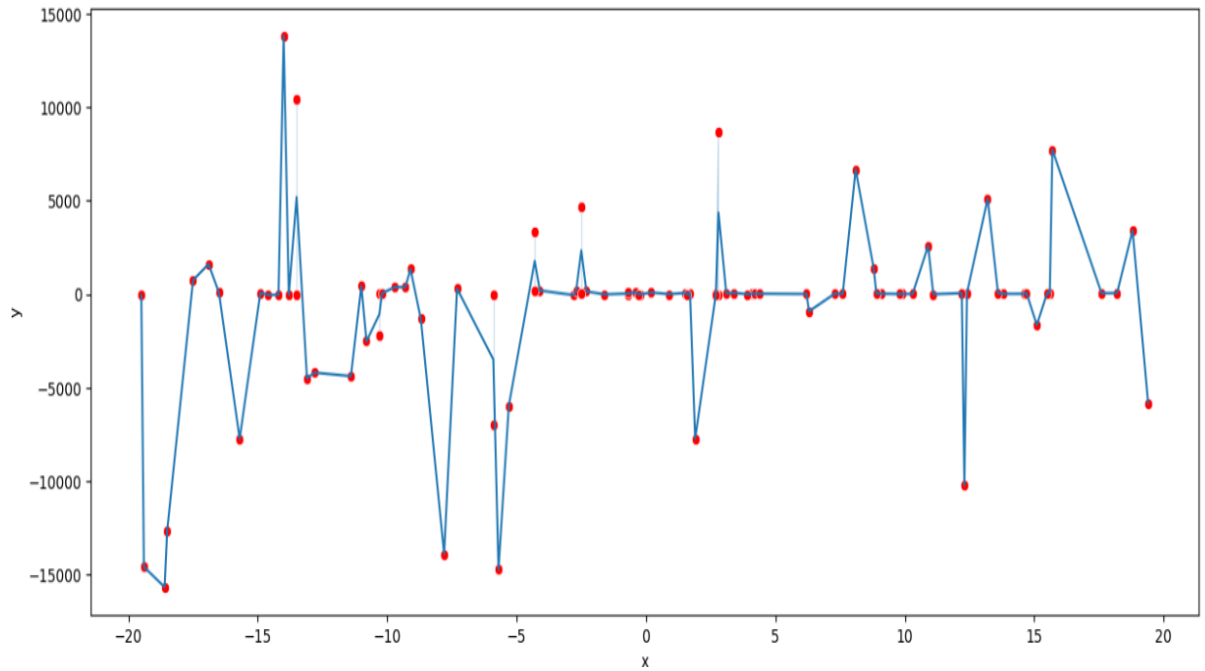
Test data frame

```
In [17]: plt.figure(figsize=(15,6))
         sns.lineplot(x=train_df['x'],y=train_df['y1'],data=train_df)
         sns.scatterplot(x=train_df['x'], y=train_df['y1'], data=train_df, color="red")
         plt.show()
```



Ideal Data frame

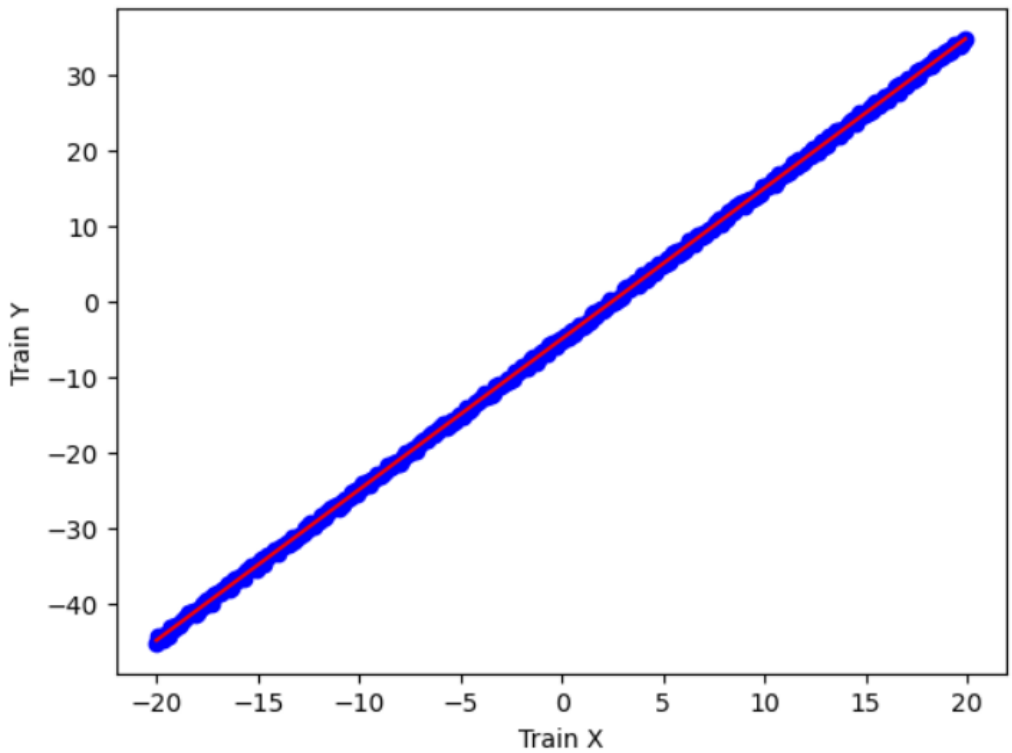
```
In [22]: plt.figure(figsize=(15,6))
         sns.lineplot(x=test_df['x'],y=test_df['y'],data=test_df)
         sns.scatterplot(x=test_df['x'],y=test_df['y'],data=test_df,color="red")
         plt.show()
```



Train Data Frame

Linear regression is carried out following the storage of the data from the CSV file in the data frame. The value of the slope and the intercept is found out that is 1.99720057 and -4.97835704 respectively. Then the prediction is calculated which comes out to be 10.99924753

```
Coefficient/slope: [[1.99720057]]
Intercept: [-4.97835704]
```




```
Predicted from [[8]] is [[10.99924753]].
```

CODING :

```
import pandas as pd
import sqlalchemy as db
from sqlalchemy import create_engine
import sqlite3 as sql
from sklearn import linear_model
import os
import matplotlib.pyplot as plt
import numpy as np

def regression(test_data, reg):
    print("Predicted from {} is {}.".format(test_data, reg.predict(test_data)))

def main():
    #Error handling when creating the engine to SQLite
    try:
        engine = create_engine('sqlite:///test.db', echo=False)
    except:
        print("Failed to create engine.")

    #Read data from csv file and store it to dataframe
    train_df = pd.read_csv(os.path.join(os.getcwd(), "train.csv"))
    #Create new table in SQLite based on dataframe
    train_df.to_sql('train_table',con=engine, index=False, if_exists='replace')

    #regress to train the data
    train_x = np.asanyarray(train_df[['x']])
    train_y = np.asanyarray(train_df[['y1']])
    global reg
    reg = linear_model.LinearRegression()
    reg.fit(train_x, train_y)

    coef = reg.coef_
    intercept = reg.intercept_

    print('Coefficient/slope: {}'.format(coef))
    print('Intercept: {}'.format(intercept))

    #plotting the training data with regression result
    #plt.scatter(train_df.iloc[:,0], train_df.iloc[:,1], color='blue')
    #plt.plot(train_x, coef * train_x + intercept, color='red')
    plt.plot(train_df.iloc[:,0], train_df.iloc[:,1], 'o', color='blue',
markersize=8, label='Data points')
```

```
    plt.plot(train_x, coef * train_x + intercept, '-', color='red',
label='Regression line')
    plt.xlabel('Train X')
    plt.ylabel('Train Y')
    plt.show()

    #regression to predict - best candidate for the function
    test_data = [[8]]
    regression(test_data,reg)

if __name__ == "__main__":
    main()
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

train_df=pd.read_csv('train.csv')
test_df=pd.read_csv('test.csv')
ideal_df=pd.read_csv('ideal.csv')

train_df

test_df

ideal_df

plt.figure(figsize=(15,6))
sns.lineplot(x=train_df['x'],y=train_df['y1'],data=train_df,palette='hls')
sns.scatterplot(x=train_df['x'],y=train_df['y1'],data=train_df,palette='hls',colo
r="red")
plt.show()

plt.figure(figsize=(15,6))
sns.lineplot(x=test_df['x'],y=test_df['y'],data=test_df,palette='hls')
sns.scatterplot(x=test_df['x'],y=test_df['y'],data=test_df,palette='hls',color="r
ed")
plt.show()
```

CONCLUSION

Utilizing training and testing data with 50 predetermined ideal functions, we performed the work using Python software. The act is carried out on four variables that are defined by the specified test functions X and Y. In addition to using a scikit-learn plot to show the relationship between the train and test datasets, we also do a linear regression model analysis on the training data set.

REFERENCE

- Semeraro, R., & Magi, A. (2019, April 16). PyPore: a python toolbox for nanopore sequencing data handling. Bioinformatics, 35(21), 4445–4447.
- Link: https://doi.org/10.1093/bioinformatics/btz269
- Sevinchan, Y., Herdeanu, B., & Traub, J. (2020, August 23). dantro: a Python package for handling, transforming,and visualizing hierarchically structured data. *Journal of Open Source Software*, *5*(52), 2316.
- Link: https://doi.org/10.21105/joss.02316

GITHUB